

TimeAudit — 你这台 Windows 电脑的"黑匣子"

一个 7×24 小时在后台默默运行的个人遥测系统：每 3 秒给你的电脑拍一张"全身 X 光"，把"哪个窗口在用、每个进程吃了多少 CPU/显卡/内存/硬盘/网络、整机温度功耗帧率、哪个进程刚出生/刚崩溃"全部记进数据库，然后用网页大盘（Grafana）回放出来。

这份 README 写给两类读者：

- 人类开发者**：哪怕你没接触过这个项目，也能在 20 分钟内搞懂它"是什么、怎么转、每个文件干嘛"。
- AI 维护者**（比如接手改代码的 Claude）：文末有"关键不变量 / 千万别踩的坑 / 测试入口"，先读那部分再动手。

想直接装起来 / 换电脑 / 灾后恢复？看隔壁的 [快速部署.md](#)。

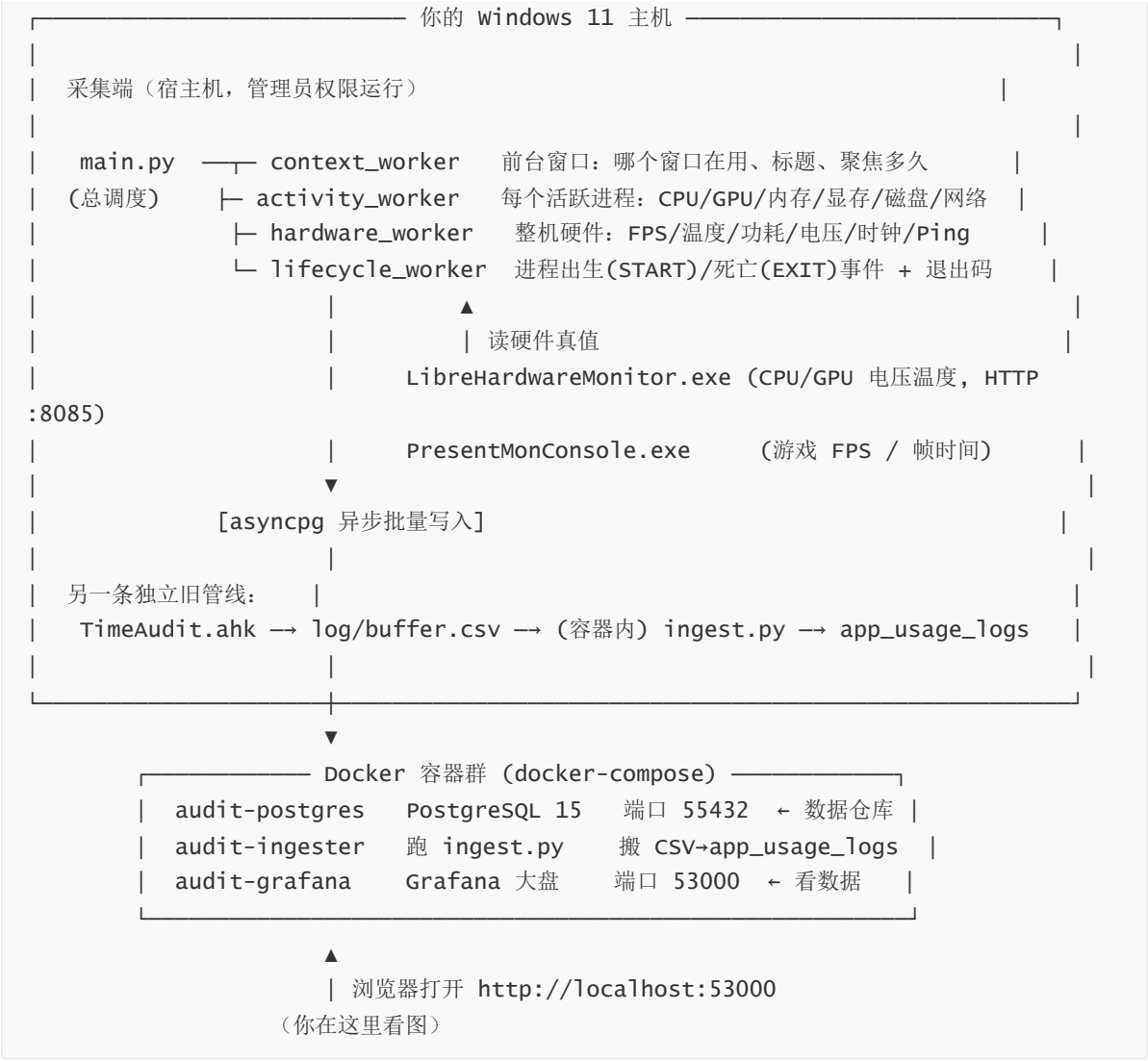
1. 一句话：它解决什么问题？

普通的任务管理器只能看"此刻"，看完就没了。**TimeAudit 的核心是"留底"+"回放"**——它把你电脑每一秒的状态都存下来，于是你可以**事后**回答这些问题：

你想知道的事	TimeAudit 怎么回答
"刚才打游戏突然卡了一下，到底谁干的？"	把时间轴拖回那 2 秒，看那一刻是哪个后台进程突然狂读硬盘 / 网络丢包 / GPU 撞了功耗墙。
"我的硬盘半夜一直在响，是谁？"	查 <code>is_foreground=0</code> 的后台进程，按磁盘读写速率排序，凶手立现。
"几十个 svchost.exe 哪个在吃 CPU？"	每个 svchost 都还原出了它背后真正的服务名（ <code>service_name</code> ）。
"这个进程是不是病毒伪装的系统文件？"	看 <code>signature_status</code> （数字签名）和 <code>command_line</code> （完整启动命令）。
"我今天到底在哪个软件上花了最多时间？"	前台聚焦时长（ <code>duration_ms</code> ）按 App 汇总。
"通宵跑 AI 训练耗了多少电？"	对 <code>gpu_board_power</code> + <code>cpu_package_power</code> 按时间积分。
"那个程序无缘无故闪退，死因是什么？"	查它的 <code>EXIT</code> 事件里的退出码（如 <code>0xc0000005</code> = 内存非法访问）。

适用场景：个人的高配工作站（本项目就是为 Windows 11 + AMD 9950X3D + RTX 5080 这套机器调的），想长期、细粒度地审计自己电脑的性能与时间花销。它**不是**给公司批量部署的监控（没有多机、没有告警推送），就是一个硬核极客的"自己电脑的飞行记录仪"。

2. 整体怎么转？（数据从哪来，到哪去）



一句话总结：采集端（Python）每 3 秒采一拍 → 直接写进 Docker 里的 PostgreSQL → 你用浏览器开 Grafana 看图。

3. ⚠ 一个容易搞混的点：这里其实有"两条"数据管线

项目历史上长出了两套前台记录，它们同时在跑、互不干扰，新手最容易在这里懵：

	管线 A：旧版「简版工时」	管线 B：主引擎「全量遥测」
谁采集	TimeAudit.ahk (AutoHotkey 脚本)	main.py + 5 个 worker (Python)
采什么	只采"前台哪个窗口、用了多久"，自带空闲/睡眠/锁屏判定	前台窗口 + 全部进程指标 + 整机硬件 + 进程生死
怎么落库	先写 log/buffer.csv，再由容器里的 ingest.py 每 10 秒搬进数据库	Python 直接异步写数据库
进哪张表	app_usage_logs (一张简单表)	fact_* / dim_* 一套分区事实表

你真正要看的、信息量最大的，是管线 B（fact_* 表）。管线 A 是更早的轻量版本，留着兜底/对照。本 README 后面讲的"采集舱""数据表"主要指**管线 B**。

4. 采集端：5 个 worker 各管一摊

主程序 `main.py` 是"总指挥"：它建数据库连接池、把 4 个采集 worker 拉起来，然后进入一个**每 3 秒一拍**的主循环，
每拍让各 worker 采一次数据并批量写库。下面逐个说人话。

`main.py` — 总调度 + 守护外壳

- 每 3 秒一拍驱动所有采集。
- **单例锁**：保证全机只有一个引擎在跑（新实例会抢占踢掉旧的）。
- **崩溃自愈外壳**：主循环若整个崩了，外层 `while True` 会等 5 秒重启它。
- **睡眠/唤醒处理**（重点，见第 7 节）：用"墙上时间"判断系统是否刚从睡眠/休眠醒来，醒来后把跨睡眠的脏数据截断掉。
- **冷启动清理**：每次启动先把上次"关机时没来得及收尾"的前台会话补上结束时间（否则会留下永不结束的"幽灵行"）。
- **分区预热**：每 12 小时（按墙上时间）提前把"下一周/下一月"的数据库分区建好，免得到了周一零点没表可写而丢数据。
- **日志治理**：`telemetry.log` 超过 50MB 自动清空截断。

`context_worker.py` — 前台上下文舱

- 用 Win32 API 高频问："现在最前面的窗口是谁、标题是什么、是全屏还是窗口"。
- 窗口一换，就把上一个窗口的会话"结算"：写下它聚焦了多少毫秒（`duration_ms`）。
- 写进 `fact_process_context`。

`activity_worker.py` — 活跃进程舱

- 用 NTDLL 一次性拿到全系统进程快照（比逐个 `psutil` 快得多），算出每个进程这一拍的 CPU、磁盘读写、IOPS（都是"速率"，靠两拍差分算）。
- GPU 显存/占用走 Windows 图形内核的 PDH 计数器（和任务管理器同源），并只认 RTX 5080 这张独显（按厂商 ID 锁 LUID，隔离核显和虚拟显示器，见第 7 节）。
- 网络流量按"每进程连接数占比"近似分摊（这是个已知的粗略估算，不是精确值）。
- 写进 `fact_process_activity`，并维护进程身份维度表 `dim_process_registry`。

`hardware_worker.py` — 整机硬件舱

- NVML 读 GPU：利用率、温度、功耗、显存时钟、PCIe、降频原因。
- PDH 读 CPU：频率、ACPI 温度、硬缺页等。
- **LibreHardwareMonitor**（外部 exe）通过 HTTP `http://127.0.0.1:8085/data.json` 读 NVML/PDH 给不出来的真值：CPU 核心电压(Vcore)、CPU 封装温度(Tctl/Tdie)、GPU 核心电压、GPU 热点温度。
- **PresentMonConsole**（外部 exe）抓游戏的 FPS / 帧时间 / 1% Low。
- 自己测 DPC 延迟、Ping、丢包、抖动。
- 这两个外部 exe 都有**看门狗**：进程没了自动隐身重启；LHM 的网页服务卡死了也会强杀重拉。
- 写进 `fact_system_hardware`。

lifecycle_worker.py — 进程生死舱

- 每秒对全系统进程做一次"差分"：这一秒多出来的就是"出生(START)"，少掉的就是"死亡(EXIT)"。
- 进程出生时就提前抓住它的内核句柄，这样它死的时候才能读到**真实退出码**（如 0xc0000005）和存活时长。
- 顺带做"数字签名校验"（判断是不是微软官方签名）和"是否提权"。
- 写进 `fact_process_lifecycle_events`。

5. 数据库里有哪些表？（每张表存什么，大白话）

数据库名 `time_audit`，跑在 Docker 容器 `audit-postgres` 里，宿主机端口 **55432**。

表名	类型	存什么	怎么分区
<code>dim_process_registry</code>	维度表	每个"独一无二的程序身份"一行：进程名+路径+命令行+父进程+是否提权+签名状态。事实表用整数 <code>process_key</code> 指向它，省空间。	不分区
<code>fact_process_activity</code>	事实表	每 3 秒 × 每个活跃进程一行：CPU/GPU/内存/显存/磁盘/网络/线程数等。	按周
<code>fact_process_context</code>	事实表	前台窗口会话：哪个窗口、标题、 <code>duration_ms</code> （聚焦多久）。	按周
<code>fact_system_hardware</code>	事实表	每 3 秒一行整机硬件画像：FPS、CPU/GPU 温度功耗电压时钟、内存、磁盘延迟、Ping。	按月
<code>fact_process_lifecycle_events</code>	事实表	进程 START/EXIT 离散事件 + 退出码 + 存活秒数。	不分区
<code>app_usage_logs</code>	旧版表	管线 A（AHK→CSV）的简版前台工时。	不分区

几个关键字段的含义（看图时会用到）：

- `proc_cpu_usage`：整机口径 0-100%（已按 32 个逻辑核归一化）。不是单核百分比，所以一个多线程进程也不会超过 100%。
- `signature_status`：1=有效数字签名（多半是正经软件），0=没签名（可疑），-1/-2=校验出错。
- `is_elevated`：1=管理员权限运行，0=普通，-1/-2=查不到。
- `window_mode`：2=全屏/无边框，3=普通窗口。

- `gpu_throttling_reasons`：GPU 降频原因的二进制位（撞功耗墙/温度墙等）。
- `duration_ms`：前台窗口连续聚焦的毫秒数。**注意**：跨睡眠/锁屏的会话会被引擎截断，不会把睡觉时间算成"在用"。

完整建表语句见 [schema.sql](#)（全新装机时用它建表）。

6. 存储 + 展示：Docker 那三个容器

`docker-compose.yml` 定义了 3 个容器（开机由 `start_all.bat` 里的 `docker compose up -d` 拉起）：

容器	镜像	端口	干嘛
<code>audit-postgres</code>	<code>postgres:15-alpine</code>	<code>55432→5432</code>	数据仓库。数据存在宿主机 <code>./postgres_data</code> 目录。
<code>audit-ingester</code>	本地 Dockerfile 构建	无	跑 <code>ingest.py</code> ，每 10 秒把 <code>log/buffer.csv</code> 搬进 <code>app_usage_logs</code> 。
<code>audit-grafana</code>	<code>grafana-oss:latest</code>	<code>53000→3000</code>	网页大盘。配置从 <code>./grafana_provisioning</code> 注入，数据存 <code>./grafana_data</code> 。

账号/端口速查：

- PostgreSQL： `localhost:55432`，用户 `leyang`，密码 `SecurePassword123`，库 `time_audit`。
- Grafana：浏览器开 `http://localhost:53000`。

7. 关键设计 & 千万别踩的坑（给 AI 维护者重点看）

这一节是整个项目最值钱的部分——很多写法是故意这样写的，看着别扭但有血泪原因。改代码前务必读完。

- 睡眠/关机边界，一律用"墙上时间" `time.time()`，绝不用"单调时钟"。**
`asyncio` 的事件循环时钟（单调时钟 `monotonic`）在系统睡眠时会暂停，所以"靠单调时钟跳变来判断唤醒"是永远不会触发的死代码。
现在 `main.py` 用墙上时间差（超过 60 秒）判断刚睡醒，醒来后会：① 把当前前台会话按"睡前那一刻"截断（否则你合盖睡 8 小时，醒来会被算成"看了这个文档 8 小时"——这是真实发生过的 24 小时脏数据）；② 重置网络/CPU 速率基线；③ 强制补建分区。
- 网络/CPU/IO 的"速率"用单调时钟 `time.monotonic()`，免疫 NTP 对时回拨。**
墙上时间偶尔会被 NTP 往回拨几秒，如果用它算速率，分母会变成极小值导致瞬间几千倍的假尖刺。所以**速率算分母**用单调时钟，而**落库时间戳**用墙上时间（UTC）。两者分工不能混。
- 前台会话时长 `duration_ms` 永远不为负。** 不管是正常切窗还是睡眠截断，结束时间若早于开始时间，一律收敛成 0。
- `fact_process_context` 的 UPDATE 必须带 `timestamp` 主键。**
闭合前台会话的 UPDATE 语句 WHERE 里一定要带分区键 `timestamp`，否则会扫遍所有周/月分区（慢），而且 Windows 复用 PID 时还可能误闭合几个月前的旧行。

5. **每进程 CPU 归一化到 0-100%**（除以逻辑核数并封顶）。不要改回"单核百分比"，否则多线程进程会冒出 2500% 这种吓人的值。
6. **GPU 只认 RTX 5080 这张独显。** 这台机器有三个显示适配器（RTX5080 独显 + AMD 核显 + 向日葵虚拟显示器）。核显是 UMA 架构会把系统内存误报成几十 GB"专用显存"。所以开机时从注册表按厂商 ID(NVIDIA=0x10DE) 锁定独显的 LUID 前缀，之后每进程 GPU 数据只认这张卡。
7. **NVML 的每个易失败调用要各自隔离。**
降频原因、PCIe 吞吐这些调用在某些驱动版本会抛异常；它们各自包了 try，单个失败不会把整块 GPU 采集拖垮、更不会触发 `nvmlShutdown` 把 GPU 数据全部清零。（注：throttle 接口在本机 RTX5080 上实测是支持的，不会崩。）
8. **两个外部 exe 必须保持在线，靠"看门狗重拉"而不是"降级造假"。**
LibreHardwareMonitor / PresentMon 掉了就自动隐身重启；LHM 网页服务卡死连续 ~15 秒也强杀重拉。理念是"要么采到真值，要么重启再采，绝不写假数据"。
9. **外部 exe 需要管理员权限。** 非提权环境下 PresentMon/LHM 会报 `WinError 740`，引擎会退避重试而**不会崩**，但拿不到 FPS/电压。所以**引擎必须以管理员身份运行**（开机自启任务已配好提权，见快速部署.md）。
10. **日志会自动控制大小：** `telemetry.log` 超 50MB 截断；`presentmon_debug.log` 用滚动文件处理器封顶 ~15MB。
11. **进程差分扫描器的快照推进放在 finally 里：** 即使处理某个进程时抛异常，也保证基线快照前进，不会把同一个 START/EXIT 事件重复投递。

8. 怎么跑起来 / 怎么看数据 / 怎么排查

完整安装、换机、容灾步骤在 [快速部署.md](#)。这里只列日常最常用的几条。

看实时日志（引擎在干嘛、谁出生谁死亡）：

```
Get-Content -Wait -Tail 20 E:\TimeAudit\telemetry.log -Encoding utf8
```

一键体检（强烈推荐排查问题时先跑它，会逐项检查：组件存活 / 真值入库 / CPU 归一化 / 显存锁卡 / 分区建表 / 数据质量）：

```
python E:\TimeAudit\test_telemetry_health.py
```

跑单元测试套件（改完代码后回归，应 6/6 全绿）：

```
python E:\TimeAudit\telemetry_test_suite.py
```

手动重启引擎（用配好的提权计划任务，最干净）：

```
schtasks /run /tn TimeAudit_AutoStart
```

手动全量备份（数据库 + Grafana 仪表盘）：

```
powershell -ExecutionPolicy Bypass -File E:\TimeAudit\backup_all.ps1
```

数据库和 Grafana 仪表盘每天凌晨 4 点会由计划任务 `TimeAudit_DailyBackup` 自动备份，仪表盘还会自动 commit + push 到 GitHub。无需手动导出。详见[快速部署.md](#)。

看大盘：浏览器开 `http://localhost:53000`。

9. 目录结构

E:\TimeAudit\ ├─ main.py	总调度 + 3 秒主循环 + 睡眠/分区/自愈
├─ context_worker.py	前台窗口上下文舱 → <code>fact_process_context</code>
├─ activity_worker.py	活跃进程舱 → <code>fact_process_activity</code> (+
<code>dim_process_registry</code>)	
├─ hardware_worker.py	整机硬件舱 → <code>fact_system_hardware</code>
├─ lifecycle_worker.py	进程生死舱 → <code>fact_process_lifecycle_events</code>
├─ TimeAudit.ahk	旧版前台工时脚本（管线 A）→ <code>log/buffer.csv</code>
├─ ingest.py	容器内：把 <code>buffer.csv</code> 搬进 <code>app_usage_logs</code>
├─ docker-compose.yml	Postgres + Ingester + Grafana 三容器编排
├─ Dockerfile	Ingester 容器构建
├─ schema.sql	【全新装机用】建好所有父表+索引
├─ requirements.txt	宿主机 Python 依赖 (<code>psutil</code> / <code>nvidia-ml-py</code> / <code>asyncpg</code>)
├─ backup_all.ps1	一键全量备份(库+大盘)；每日计划任务调用它
├─ backup_db.ps1	备份数据库 (<code>pg_dump -Fc</code> + 自动清旧)
├─ backup_grafana.py	备份 Grafana 仪表盘(导出JSON+git push) + <code>grafana.db</code>
├─ restore_grafana.py	从备份 JSON 恢复 Grafana 仪表盘
├─ start_all.bat	开机自启：拉起 AHK + Docker + 提权的 <code>main.py</code>
├─ check_status_gui.ps1	图形化状态体检小工具
├─ test_telemetry_health.py	在线健康综合体检（先跑它排查问题）
├─ telemetry_test_suite.py	单元/集成测试套件（6 个用例）
├─ LibreHardwareMonitor.exe	外部硬件探针（CPU/GPU 电压温度，HTTP :8085）
├─ PresentMonConsole.exe	外部帧率探针（FPS / 帧时间）
├─ postgres_data/	PostgreSQL 数据卷（别手删！）
├─ grafana_data/	Grafana 数据卷（含 <code>grafana.db</code> 仪表盘库）
├─ grafana_provisioning/	Grafana 数据源/大盘 <code>provider</code> 配置（自动注入容器）
├─ backups/	数据库 <code>.dump</code> 备份（不进 git）
├─ grafana_backups/	<code>dashboards/*.json</code> (进 git) + <code>grafana_db/*.db</code> (不进 git)
├─ log/	AHK 的 <code>buffer.csv</code> + 备份日志 <code>backup.log</code>

10. 给 AI 维护者的快速上手清单

1. 先读第 7 节"关键设计 & 坑"，那里是历史血泪，照着它就不会把修好的 bug 改回去。
2. 改完代码先跑两件事：`python telemetry_test_suite.py`（应 6/6 绿）+ `python test_telemetry_health.py`（引擎在跑时应大部分绿）。
3. 跑测试要用 UTF-8：默认 GBK 控制台会因日志里的 emoji 崩；套件已自愈 stdout，但你手动跑别的脚本时记得 `PYTHONUTF8=1`。
4. 这是写密集时序库：每 3 秒几十行写入。加索引、改表结构前先想清楚写放大代价。

5. **本机就是目标机** (RTX 5080 + 9950X3D, PostgreSQL 跑在 55432) 。很多硬件相关逻辑可以直接实机验证, 别只靠推演。
6. **测试里 mock NVML 时, 假句柄(MagicMock)千万别传给真实 NVML 函数**——会在 C 层访问违例直接把进程干崩 (try/except 拦不住) 。所有 NVML 函数名都要 mock 到位。

TimeAudit 是个人项目。数据全部留在本机, 不上传任何云端。